CS 351 | NATURAL LANGUAGE PROCESSING

When Does Neural Memory Fail? Investigating Distractors in Multi-Hop Reasoning

Project blueprint • Research rationale, technical architecture, controlled experiment and four-week execution plan

Team: Bjibji Hamza · Elbouk Abderrahimane

Project type: Direction 1 — understand a published model by implementing its central mechanisms

Status: Proposed plan, not experimental results | October 2026

One-sentence vision. Build a small question-answering neural network from scratch, then investigate whether repeated attention helps it recover facts when irrelevant sentences crowd its memory.

1. Executive overview

We will reproduce a compact End-to-End Memory Network (MemN2N) inspired by Sukhbaatar et al. (NeurIPS 2015). The network receives a short English story and a question, stores the story sentences as learned vector representations, attends to those memory slots one or more times, and predicts an answer. Unlike an API-based chatbot, the central architecture will be our own PyTorch implementation.

Research question: How does the number of memory hops influence multi-fact question answering as irrelevant, answer-preserving sentences are added to the story?

Hypothesis: Three hops may better support questions requiring several facts and may degrade less sharply under distractors than one hop. This is a prediction to test; no advantage is assumed.

2. What the system will actually do

Input story: Mary picked up the apple. John went to the bathroom. Mary traveled to the kitchen. Daniel entered the garden. Mary dropped the apple.

Question: Where is the apple? Target answer: kitchen.

The training objective is to predict the target answer from the story and question. A trained model is not explicitly given which sentence to attend to; attention patterns emerge through answer supervision.

3. Research literature and scientific context

Primary paper — End-To-End Memory Networks (2015)

Sukhbaatar, S., Szlam, A., Weston, J., and Fergus, R. (2015), End-To-End Memory Networks, NeurIPS. The paper introduces multiple differentiable attention-based reads over external memory and trains the network end-to-end. Our implementation will focus on the supervised question-answering setting rather than reproducing the complete paper.

Benchmark paper — Towards AI-Complete Question Answering (2015)

Weston, J. et al. (2015) introduced the synthetic bAbI tasks to test specific reasoning skills, including one-, two-, and three-supporting-fact question answering. These tasks offer controlled examples, but do not establish general real-world reasoning ability.

What is original in our student project?

The underlying architecture and multi-hop comparison are established research. Our contribution is a focused, controlled robustness study: insert semantically irrelevant, answer-preserving facts at several levels and measure the accuracy loss, attention changes, and qualitative failures. This is an experimental extension, not a claim to have invented a new model.

Key limits and claims to avoid

• Attention weights show how the network distributes retrieval weight, but are not definitive explanations of human-like reasoning.

• Synthetic bAbI stories have a narrow vocabulary and grammar. Accuracy here does not imply competence on open-domain QA.

• Additional hops increase computation; their benefit depends on the task, training stability, and evaluation setting.

4. Connection to the CS 351 syllabus

Course material	Concrete project usage
Week 2 — tokenization / embeddings	Token IDs, vocabulary, learned question and memory embeddings
Week 3 — word representations	Embedding matrices learned with supervision
Week 4 — statistical NLP	Simple bag-of-words question-answering baseline
Week 5 — neural sequence learning	Trainable neural architecture, gradient descent, sequence context (not an LSTM reproduction)
Week 6 — attention	Similarity, softmax, attention-weighted memory reads
Week 7 — Transformers	Compare concepts with attention; no Transformer blocks are implemented
Weeks 8–10 — LLM / RAG	Conceptual links only; no pretrained LLM, RAG pipeline, or instruction tuning

5. Architecture: from tokens to answer

Stage	Implementation	Expected tensor / operation
1. Tokenization	Lowercase / tokenize; map words to training vocabulary	IDs for question and sentences
2. Memory encoder	Trainable embedding table A; aggregate tokens in each sentence	Memory keys m1 ... mn
3. Question encoder	Embedding matrix B; aggregate question tokens	Query vector u
4. Attention	Similarity between query and every key; softmax	p_i = softmax(u · m_i)
5. Memory read	Weighted sum of output memory vectors from matrix C	o = sum_i p_i c_i
6. Next hop	Update state and repeat attention	u_next = u + o (basic variant)
7. Classifier	Linear output head and softmax over answer vocabulary	Predicted answer token

Engineering boundary: PyTorch supplies tensors, embedding layers, automatic differentiation, optimizer, and loss. We implement the memory organization, multi-hop computation, masking, training loop, evaluation, and attention extraction. We do not reuse pretrained QA weights.

Implementation choices to decide early

• Use embedding size 32 initially; increase to 64 only if necessary.

• Handle padded sentences and padded memory slots explicitly; mask padded memories before softmax.

• Choose a consistent way to encode sentence order. Preserve chronology because answers often depend on the last relevant event.

• Restrict output labels to answers found in training vocabulary; report any out-of-vocabulary cases.

• Unit-test that a forward pass returns expected shapes and that gradients reach every embedding matrix.

6. Data and preprocessing

Dataset: Meta/Facebook bAbI QA, hosted on Hugging Face, English synthetic stories. Use configurations en-valid-10k-qa1, en-valid-10k-qa2, and en-valid-10k-qa3, with approximately 1,800 training, 200 validation, and 200 test story records per task. Each record may contain multiple questions, so the number of question-level supervised examples is different and must be reported after extraction. License: CC BY 3.0; provide attribution.

Extraction pipeline: Walk each story chronologically; whenever a question is encountered, create one (preceding context, question, answer) example. Keep the source record ID, task ID, and supporting-fact IDs for inspection. Never allow future story lines to enter an earlier question context.

Leakage safeguards: Respect official splits, do not construct vocabulary or tune parameters from the test set, and check duplicate question/context examples across splits. Keep related questions from a source story together.

7. Controlled experimental design

Baseline and models

System	Purpose	Training
BoW QA classifier	Simple reference with story/question features but no explicit iterative memory	Train from scratch
MemN2N — 1 hop	Single attention-based read	Train from scratch
MemN2N — 3 hops	Repeated attention-based reads	Train from scratch

Independent variables and controls

Main architectural variable: number of hops (1 vs 3). Test-time stress variable: number of irrelevant sentences inserted into the story (0, 5, 10, 20). Other settings (data, embedding dimension, optimizer, budget, preprocessing) remain fixed as far as possible. Train on unmodified official training contexts; evaluate on the original and modified test contexts. Consider matched seeds and repeated runs if time permits.

Distractor validity: Insert only facts that do not modify the location, ownership, or identity needed for the correct answer. Avoid referencing the queried entity or its chain of relevant actors. Test insertions at fixed/randomized positions and document the procedure. Verify the target answer is unchanged using rules and spot checks. Increase maximum memory capacity to hold all retained facts; do not silently truncate supporting facts.

Evaluation and visuals

• Exact-match answer accuracy for QA1, QA2, QA3, with task-wise model comparisons.

• Accuracy vs number of distractors; report absolute and relative performance drops from clean testing.

• Attention heatmaps across memory slots and hops for selected correct and incorrect predictions.

• Manual error cases: wrong last-known location, failure to chain facts, distractor attention, or padding/truncation issues.

• Training/validation learning curves, random seeds, and configuration details for reproducibility.

Interpretation caution: A difference between 1-hop and 3-hop models may reflect optimization or parameter differences as well as reasoning capacity. We will state those limitations and avoid strong causal claims beyond the controlled comparison.

8. What we will deliver

Artifact	Description
Code repository	Modular preprocessing, models, training, evaluation, configs, README
Model checkpoints	Small saved models and deterministic inference script
Results package	Metrics CSV files, training plots, robustness curves
Attention explorer	Lightweight notebook UI: story + question + hop-by-hop attention
Technical report	Literature, methods, experiments, errors, limitations
Presentation	Paper mechanisms, architecture diagram, result plots, live examples

9. Four-week implementation roadmap

Week	Tasks	Definition of done
1 — Foundations	Read paper Sections 1–2; load bAbI; convert stories to question examples; build vocabulary, splits and BoW baseline	Dataset reproducible; baseline trained and evaluated
2 — Core model	Implement embeddings, memory slots, masked attention, 1-hop forward pass; debug loss and tiny-batch overfitting	1-hop MemN2N trains and passes sanity checks
3 — Controlled study	Add 3 hops; train with matched settings; generate validated distractors; run clean/stress tests	Accuracy tables and distractor comparisons saved
4 — Research output	Analyze examples; extract attention maps; implement simple viewer; finish report and slides	Reproducible repository and final demo

Team responsibilities (proposed)

Bjibji Hamza: literature review, architecture design, PyTorch MemN2N implementation and training checks. Elbouk Abderrahimane: bAbI preprocessing, BoW baseline, distractor generator and evaluation scripts. Both: validation, model debugging, quantitative analysis, technical writing and presentation. Adjust this allocation together before submission.

10. Compute and risk control

Practical target: modern laptop CPU (4+ cores, 8–16 GB RAM), around 2–5 GB free workspace; optional Colab GPU. The selected tasks and 32-dimensional model are lightweight compared with LLMs. Exact training duration depends on implementation, number of question-level examples, epochs, seeds and CPU/GPU; benchmark one epoch before scheduling complete runs.

Suggested initial settings: 32-dimensional embeddings, batch size 32, Adam, 20–30 epoch starting budget with validation-based early stopping, models with 1 and 3 hops. These are starting points rather than guaranteed optimal values.

Fallback: If the 1-hop system is not stable by the end of Week 2, reduce to QA1 + QA2, 1 vs 2 hops, and distractor levels 0/5/10; prioritize one rigorous experiment and a clean report over extra functionality.

11. Reproducibility checklist

- Fixed seeds; saved config and package versions.
- Exact task configurations, record counts and extracted QA-example counts.
- Only validation-selected hyperparameters; untouched official test data.
- Answer-preserving perturbation rules, samples and unit tests.
- Evaluation scripts producing tables and figures from saved predictions.
- Clear separation between published findings, our hypotheses and our eventual measured results.

12. Project structure and first actions

Suggested repository layout:

README.md | requirements.txt | configs/ | data/ (ignored) | src/preprocess.py | src/baseline.py | src/memn2n.py | src/train.py | src/evaluate.py | src/distractors.py | notebooks/attention_explorer.ipynb | reports/

Initial verification tasks

- Download QA1, QA2, QA3 and print raw dataset schema, split sizes and extracted question counts.
- Write five manually checked story/question examples and verify chronological context extraction.
- Train a baseline and verify that performance exceeds a majority-class heuristic.
- Build a 1-hop MemN2N that can overfit 20 training examples before training at scale.
- Design and test a distractor generator that cannot change gold answers.

13. References and links

1. Sukhbaatar et al. (2015), End-To-End Memory Networks, NeurIPS: https://arxiv.org/abs/1503.08895
2. Official NeurIPS paper listing: https://papers.neurips.cc/paper_files/paper/2015/hash/8fb21ee7a2207526da55a679f0332de2-Abstract.html
3. Weston et al. (2015), Towards AI-Complete Question Answering: https://arxiv.org/abs/1502.05698
4. Meta/Facebook bAbI dataset and license: https://huggingface.co/datasets/facebook/babi_qa
5. Course syllabus: https://anassbelcaid.github.io/nlp-course-2026/schedule.html
6. Course student-project requirements: https://anassbelcaid.github.io/nlp-course-2026/student-projects.html

Project success criterion: A correctly implemented and documented neural memory model, a fair baseline, and an evidence-based answer to whether more attention hops resist irrelevant information. High accuracy alone is not the goal.